

QualiJournal Admin – NameError 수정 및 재현 가능한 배포 런북

이 문서는 QualiJournal Admin 패널에서 `NameError: name 'kw' is not defined` 와 같은 오류가 발생하는 경우 이를 수정하고, 배포와 QA 자동 점검 흐름을 재현 가능하게 만드는 방법을 단계별로 설명합니다. 특히 `_split_kw_tokens(kw)` 호출이나 `re.sub(..., kw)` 호출에서 발생하는 `kw` 미정의 오류는 기존 패치 과정에서 변수명을 잘못 대체한 것이 원인입니다. 아래 절차는 코드 수정부터 재배포, QA 자동 점검까지 전 과정을 포함합니다.

1. 문제 원인 이해

- `orchestrator.py` 의 `build_keyword_selection` 과 `publish_keyword_page` 함수에서는 원래 로컬 변수 `kw` 를 사용하여 키워드를 정제하고 토큰화합니다. 예를 들어, 깃허브 원본 코드에서는 `kw = (keyword or "").strip()` 를 정의한 뒤 이 변수를 `_split_kw_tokens(kw)` 등에 전달합니다 ¹ .
- 패치 과정에서 `kw` 를 `keyword` 로 일괄 변경하는 바람에 위 함수들 안에서 `kw` 가 정의되지 않은 채로 사용되어 `NameError: name 'kw' is not defined` 오류가 발생했습니다 ² .
- 따라서, 함수 시그니처는 유지하되 함수 내부에서 `kw` 를 다시 정의하고 이후 코드에서 `kw` 변수를 사용하도록 수정해야 합니다.

2. 코드 수정 (패치 적용)

1. 로컬 환경에서 수정할 파일: `C:\Users\user\Desktop\퀄리저널\orchestrator.py` (또는 컨테이너의 `/app/orchestrator.py`)
2. 수정 대상 함수들:
3. `build_keyword_selection(keyword: str, cfg: dict | None = None, use_external_rss: bool = False)`
4. `publish_keyword_page(keyword: str)`
5. 수정 방법:
6. 두 함수의 시작 부분에 다음 줄을 추가하여 `kw` 를 정의합니다.

```
kw = (keyword or "").strip()
```

7. 이후 코드에서 `keyword` 대신 `kw` 를 사용하도록 원래의 로직을 복원합니다. 특히 `_split_kw_tokens(kw)` 호출, 정규식 빌더, 외부 RSS 수집기, 파일명 생성 등에 `kw` 를 넘겨야 합니다.
8. 예를 들어 `publish_keyword_page` 에서는 다음과 같이 수정합니다:

```
def publish_keyword_page(keyword: str) -> str:
    keyword = (keyword or "").strip()
    if not keyword:
        raise ValueError("키워드를 입력해 주세요.")
```

```
kw = keyword # kw를 정의해 이후에 사용할 수 있도록 함
safe_keyword = re.sub(r'[\V:*?"<>|]+' , "-", kw).strip() or "keyword"
...
```

9. 비슷하게 `build_keyword_selection` 함수의 처음에 `kw = keyword` 정의를 추가하고 `_split_kw_tokens(kw)` 등에서 사용합니다.
10. 검증: 수정 후 `python orchestrator.py --collect-keyword IPC-A-610` 을 실행해 더 이상 `NameError` 가 발생하지 않는지 확인합니다.

3. 컨테이너 재빌드 및 이미지 배포 (Plan B)

3.1 환경 변수 설정

다음 환경 변수를 설정하여 프로젝트와 서비스 이름을 정의합니다. 실제 환경에 맞게 수정할 수 있습니다.

```
$PROJECT="quali-journal-prod"
$REG_SEO="asia-northeast3" # 소스 서비스 리전
$REG_TYO="asia-northeast1" # 도메인 서비스 리전
$SRC="quali-journal-admin" # 메인 소스 서비스
$LIVE="quali-admin-domap" # 도메인 매핑 서비스
```

3.2 HTML 및 스크립트 포함 보장

컨테이너에 최신 UI와 Python 스크립트가 포함되도록 `.dockerignore` 와 `.gcloudignore` 파일에 아래 규칙이 있어야 합니다:

```
!index.html
!index_lite_black.html
!**/*.html
!*.py
!**/*.py
```

필요하다면 `orchestrator.py` , `tools/` , `feeds/` , `data/` 디렉터리를 `admin` 빌드 컨텍스트로 복사합니다.

3.3 IAM 권한 확인

`NameError` 수정 후 컨테이너 이미지 배포에서도 `Container import failed` 오류를 피하려면 다음과 같이 IAM 권한을 부여합니다.

```
# 런타임 서비스 계정들에게 이미지 읽기/비밀 읽기 권한 부여
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA_SRC" --
role="roles/artifactregistry.reader"
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA_SRC" --
role="roles/secretmanager.secretAccessor"
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA_LIVE" --
```

```

role="roles/artifactregistry.reader"
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA_LIVE" --
role="roles/secretmanager.secretAccessor"

# Cloud Build 계정에 이미지 쓰기 권한 부여
$PN=$(gcloud projects describe $PROJECT --format="value(projectNumber)")
$CB_SA="$PN@cloudbuild.gserviceaccount.com"
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$CB_SA" --
role="roles/artifactregistry.writer"

```

3.4 이미지 빌드 및 배포

1. **Artifact Registry 생성** - 리포지토리 `quali` 가 없으면 생성합니다.

```

gcloud artifacts repositories create quali --repository-format=docker --
location=$REG_SEO || true

```

2. **이미지 빌드 및 푸시** - admin 디렉터리에서 수정된 소스를 이미지로 빌드하고 Artifact Registry에 푸시합니다.

```

cd "C:\Users\user\Desktop\퀄리저널\admin"
gcloud builds submit --tag asia-northeast3-docker.pkg.dev/$PROJECT/quali/
qualijournal-admin:stable

```

3. **배포** - 빌드한 이미지를 메인 서비스와 도메인 서비스에 배포하고 트래픽을 최신 리버전으로 전환합니다.

```

$IMG="asia-northeast3-docker.pkg.dev/$PROJECT/quali/qualijournal-admin:stable"

# 메인 서비스(SRC) 배포
gcloud run deploy $SRC --image $IMG \
--region $REG_SEO --project $PROJECT \
--set-secrets ADMIN_TOKEN=ADMIN_TOKEN:latest --allow-unauthenticated \
--set-env-vars "PYTHONUTF8=1,PYTHONIOENCODING=utf-8"

# 도메인 서비스(LIVE) 배포
gcloud run deploy $LIVE --image $IMG \
--region $REG_TYO --project $PROJECT \
--set-secrets ADMIN_TOKEN=ADMIN_TOKEN:latest --allow-unauthenticated \
--set-env-vars "PYTHONUTF8=1,PYTHONIOENCODING=utf-8"

# 트래픽 최신 리버전 100 %
gcloud run services update-traffic $SRC --to-latest --region $REG_SEO --
project $PROJECT
gcloud run services update-traffic $LIVE --to-latest --region $REG_TYO --
project $PROJECT

```

3.5 헬스 체크

서비스 URL을 가져와 `/health`와 `/api/status` 엔드포인트가 200 OK를 반환하는지 확인합니다:

```
$URL=$(gcloud run services describe $LIVE --region $REG_TYO --project $PROJECT --
format="value(status.url)")

# Health 체크
curl -s -o /dev/null -w "%{http_code}\n" "$URL/health" # → 200
curl -s -o /dev/null -w "%{http_code}\n" -H "X-Admin-Token: $TOKEN" "$URL/api/status" # →
200
```

4. QA 자동 점검 시나리오 검증

1. 브라우저에서 `https://admin.standardai.co.kr/?v=<random>` (캐시 무시)로 접속한다.
2. ADMIN_TOKEN을 입력하여 인증 배지를 확인한다.
3. [QA 자동 점검] 버튼을 클릭하면 다음 순서로 로그가 출력돼야 한다:
4. `/api/flow/keyword` 호출로 키워드 수집 → 승인 Top20 → 발행 작업 수행
5. `/api/items?state=ready` 호출로 Ready 상태 목록 확인
6. 첫 Ready 항목이 있으면 `/api/items/{id}/publish` 호출로 발행
7. `/api/export/md?preview=1` 와 `/api/export/csv` 호출로 Markdown/CSV 내보내기 검증
8. 에러가 발생하면 모달 로그를 확인한다. `NameError: name 'kw' is not defined` 가 보이면 2 단계의 코드 수정이 적용되지 않은 것이므로 빌드 컨텍스트와 이미지 버전이 최신인지 다시 확인한다.
9. 성공하면 모달에 “QA 완료” 메시지가 표시되며, 가운데 영역에 `archive/..._ALL.md`, `archive/..._selected.md`, `archive/...csv` 링크가 생성된다. Export MD/CSV 파일을 내려받아 내용이 비어 있지 않은지 확인한다.

5. 결론

- `NameError` 는 패치 중 `kw` 변수가 잘못 변경된 데서 비롯된다. 함수 내부에 `kw = (keyword or '').strip()` 를 정의해 문제를 해결한다 ¹ ² .
- 수정된 파일로 컨테이너 이미지를 다시 빌드하고 배포한 후, IAM 권한과 `.dockerignore` / `.gcloudignore` 규칙을 점검한다.
- 이미지 배포 후 트래픽을 최신 리버전에 100 % 전환하고 브라우저 캐시를 무시해 새 UI를 확인한다.
- QA 자동 점검이 정상적으로 수행되면 모달 로그에 수집→승인→발행→Export 플로우가 순서대로 표시되고, Markdown/CSV가 비어 있지 않은지 검증한다.

이러한 절차를 따르면 **QualiJournal Admin**의 컨테이너 배포와 QA 자동 점검을 안정적으로 재현할 수 있으며, `NameError` 오류까지 처리하여 서비스의 신뢰성을 높일 수 있다.

¹ ² `orchestrator.py`

<https://github.com/cdmacs1003-cyber/quali-journal/blob/main/orchestrator.py>